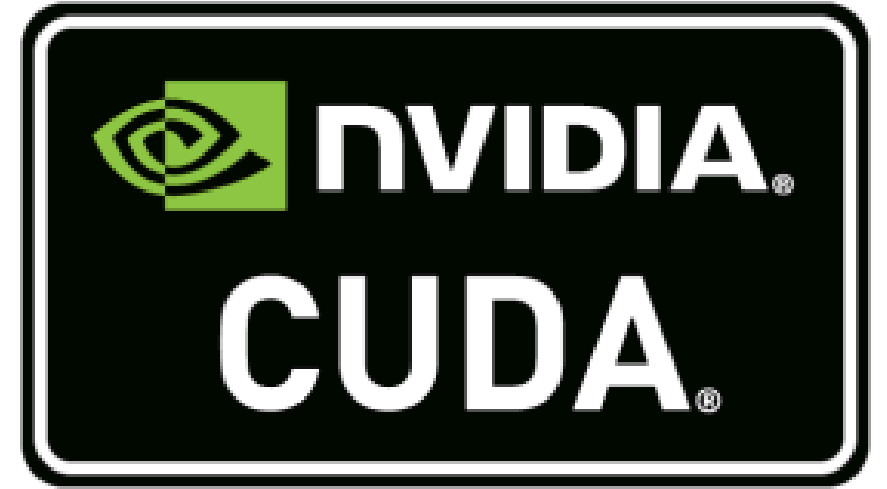




CUDA

**Неграфические параллельные вычисления
на графических процессорах**

Введение

- Вычисления на GPU развивались и развиваются очень быстро.
- Два основных производителя видеочипов, Nvidia и AMD, разработали и анонсировали соответствующие платформы под названием CUDA (Compute Unified Device Architecture) и CTM (Close To Metal или AMD Stream Computing), соответственно.
- В отличие от предыдущих моделей программирования GPU, эти были выполнены с учётом прямого доступа к аппаратным возможностям видеокарт.
- **Платформы не совместимы между собой:** CUDA — это расширение языка программирования C, а CTM — виртуальная машина, исполняющая ассемблерный код.

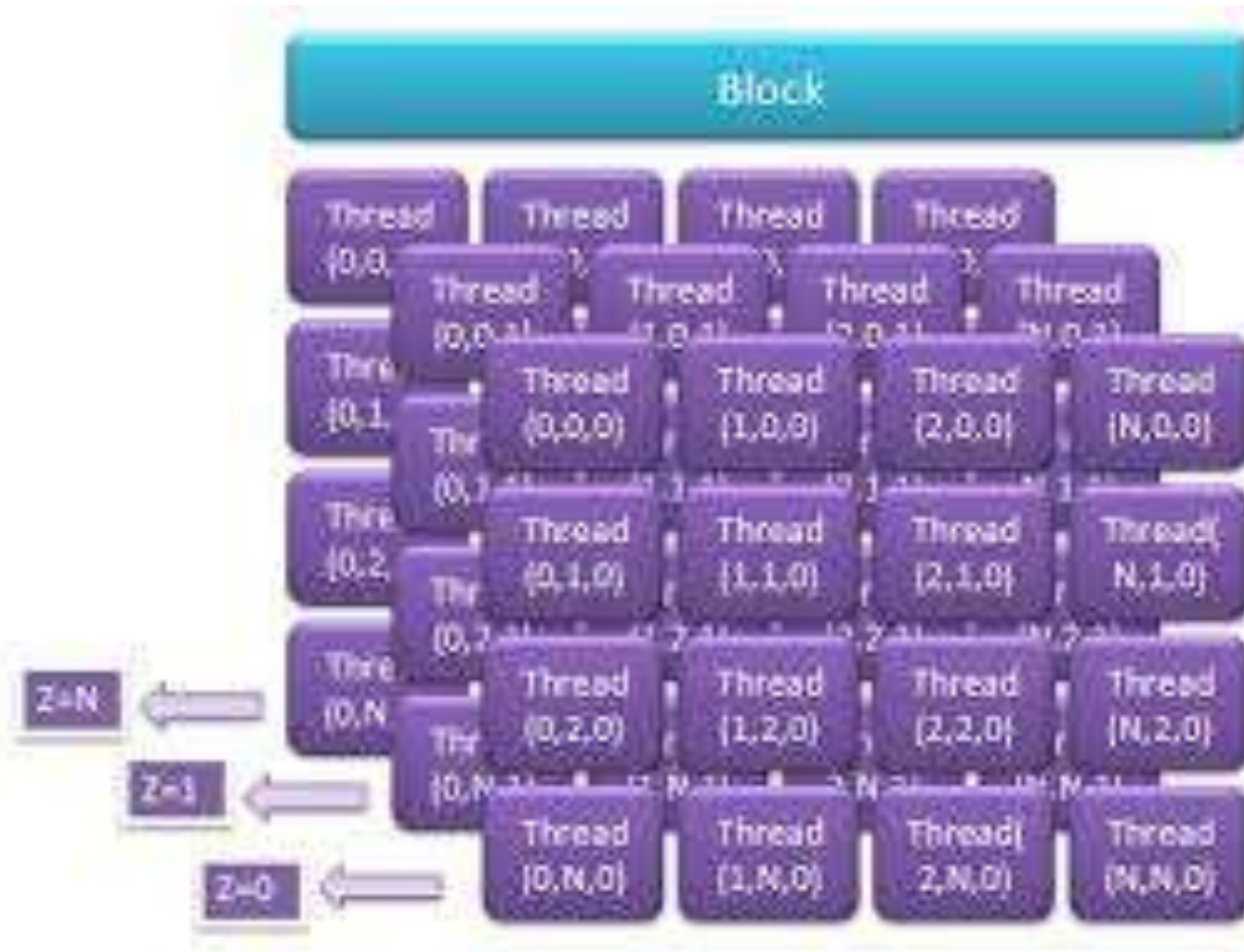
Сравнение CPU и GPU



Вычислительная модель GPU

- Внутренняя модель nVidia GPU – ключевой момент в понимании GPGPU с использованием CUDA.
- Верхний уровень ядра GPU состоит из блоков, которые группируются в сетку или грид (grid) размерностью $N1 * N2 * N3$.
- Любой блок в свою очередь состоит из нитей (threads), которые являются непосредственными исполнителями вычислений. Нити в блоке сформированы в виде трехмерного массива, размерность которого так же можно узнать с помощью функции **cudaGetDeviceProperties**, за это отвечает поле **maxThreadsDim**.

Устройство блока GPU



Базовые понятия и термины CUDA

- **Устройство (device)** — GPU. Выполняет роль «подчиненного» — делает только то, что ему говорит CPU.
- **Хост (host)** — CPU. Выполняет управляющую роль — запускает задачи на устройстве, выделяет память на устройстве, перемещает память на/с устройства. И да, использование CUDA предполагает, что как устройство так и хост имеют свою отдельную память.
- **Ядро (kernel)** — задача, запускаемая хостом на устройстве.

- При использовании технологии CUDA пишется код на своем любимом языке программирования, после чего компилятор CUDA сгенерирует **код отдельно для хоста и отдельно для устройства**.
- Небольшая оговорка: код для устройства должен быть написан только на языке C с некоторыми 'CUDA-расширениями'.

Список поддерживаемых языков:

1. C, C++
2. Common Lisp – cl-cuda
3. Fortran – FORTRAN CUDA, PGI CUDA Fortran Compiler
4. F# – Alea.CUDA
5. Haskell – Data.Array.Accelerate
6. IDL – GPULib
7. Java – jCUDA, JCuda, JCublas, JCufft, CUDA4J

Основные этапы CUDA-программы

1. Хост выделяет нужное количество памяти на устройстве.
2. Хост копирует данные из своей памяти в память устройства.
3. Хост запускает выполнение определенных ядер на устройстве.
4. Устройство выполняет ядра.
5. Хост копирует результаты из памяти устройства в свою память.

Эффективность

- Естественно, для наибольшей эффективности использования GPU нужно чтобы соотношение времени, потраченного на работу ядер, к времени, потраченному на выделение памяти и перемещение данных, было как можно больше.

Ядра

- Важный принцип — **ядра пишутся как (практически) обычные последовательные программы**. То есть создания и запуска потоков в коде самих ядер нет.
- Вместо этого, для организации параллельных вычислений **GPU запустит большое количество копий одного и того же ядра в разных потоках**. А точнее, программист определяет сам, сколько потоков запустить.
- Чем больше потоков запускается (при условии что все они будут выполнять полезную работу) — тем лучше.

Код для ядер отличается от обычного последовательного кода в таких моментах:

1. Внутри ядер имеется возможность узнать «идентификатор» или, проще говоря, позицию потока, который сейчас выполняется — используя эту позицию мы добиваемся того, что одно и то же ядро будет работать с разными данными в зависимости от потока, в котором оно запущено.
2. В некоторых случаях в коде ядра необходимо использовать различные способы синхронизации.

CUDA и язык C

Технология CUDA (компилятор nvcc.exe) вводит ряд дополнительных расширений для языка C, которые необходимы для написания кода для GPU:

- Спецификаторы функций, которые показывают, как и откуда будут выполняться функции.
- Спецификаторы переменных, которые служат для указания типа используемой памяти GPU.
- Спецификаторы запуска ядра GPU.
- Встроенные переменные для идентификации нитей, блоков и др. параметров при исполнении кода в ядре GPU .
- Дополнительные типы переменных.

CUDA C



Standard C Code

```
void saxpy_serial(int n,
                  float a,
                  float *x,
                  float *y)
{
    for (int i = 0; i < n; ++i)
        y[i] = a*x[i] + y[i];
}

// Perform SAXPY on 1M elements
saxpy_serial(4096*256, 2.0, x, y);
```

Parallel C Code

```
__global__
void saxpy_parallel(int n,
                    float a,
                    float *x,
                    float *y)
{
    int i = blockIdx.x*blockDim.x +
            threadIdx.x;
    if (i < n) y[i] = a*x[i] + y[i];
}

// Perform SAXPY on 1M elements
saxpy_parallel<<<4096, 256>>>(n, 2.0, x, y);
```

Спецификаторы функций

- **__host__** — выполнятся на CPU, вызывается с CPU (в принципе его можно и не указывать).
- **__global__** — выполняется на GPU, вызывается с CPU.
- **__device__** — выполняется на GPU, вызывается с GPU.

Спецификаторы запуска ядра

- Спецификаторы запуска ядра служат для описания количества блоков, нитей и памяти, которые вы хотите выделить при расчете на GPU. Синтаксис запуска ядра имеет следующий вид:

-

```
myKernelFunc<<<gridSize, blockSize, sharedMemSize,  
cudaStream>>>(float* param1, float* param2),
```

где

- **gridSize** – размерность сетки блоков (dim3), выделенную для расчетов,
- **blockSize** – размер блока (dim3), выделенного для расчетов,
- **sharedMemSize** – размер дополнительной памяти, выделяемой при запуске ядра,
- **cudaStream** – переменная `cudaStream_t`, задающая поток, в котором будет произведен вызов.

- Сначала задаются размеры сетки (grid), в 3D координатах: *grid_x*, *grid_y*, *grid_z*. В результате, сетка будет состоять из $grid_x * grid_y * grid_z$ блоков.
- Потом задаются размеры блока в 3D координатах: *block_x*, *block_y*, *block_z*. В результате, блок будет состоять из $block_x * block_y * block_z$ потоков.
Итого, имеем $grid_x * grid_y * grid_z * block_x * block_y * block_z$ потоков.
Важное замечание — максимальное количество потоков в одном блоке ограничено и зависит от модели GPU — типичны значения 512 (более старые модели) и 1024 (более новые модели).
- Внутри ядра доступны переменные **threadIdx** и **blockIdx** с полями **x**, **y**, **z** — они содержат 3D координаты потока в блоке и блока в сетке соответственно.
- Также доступны переменные **blockDim** и **gridDim** с теми же полями — размеры блока и сетки соответственно.

CUDA host API

- Перед тем, как приступить к непосредственному использованию CUDA для вычислений, необходимо ознакомиться с так называемым CUDA host API, который является связующим звеном между CPU и GPU.
- CUDA host API в свою очередь можно разделить на низкоуровневое API под названием CUDA driver API, который предоставляет доступ к драйверу пользовательского режима CUDA, и высокоуровневое API – CUDA runtime API.

В CUDA runtime API входят следующие группы функций:

- **Device Management** – включает функции для общего управления GPU.
- **Thread Management** – управление нитями.
- **Stream Management** – управление потоками.
- **Event Management** – функция создания и управления событиями.
- **Execution Control** – функции запуска и исполнения ядра CUDA.
- **Memory Management** – функции управлению памятью GPU.
- **Texture Reference Manager** – работа с объектами текстур через CUDA.
- **OpenGL Interoperability** – функции по взаимодействию с OpenGL API.
- **Direct3D 9 Interoperability** – функции по взаимодействию с Direct3D 9 API.
- **Direct3D 10 Interoperability** – функции по взаимодействию с Direct3D 10 API.
- **Error Handling** – функции обработки ошибок.

Пример использования CUDA

- **Задача.** Требуется вычислить сумму двух векторов размерностью N элементов.

Нам известна максимальные размеры блока: $512 \times 512 \times 64$ нитей. Так как вектор в задаче одномерный, то ограничимся использованием одного x -измерения блока, то есть задействуем только одну полосу нитей из блока.

- Заметим, что x -размерность блока 512, то есть, мы можем сложить за один раз векторы, длина которых $N \leq 512$ элементов.

В программе необходимо выполнить следующие этапы:

1. Получить данные для расчетов.
2. Скопировать эти данные в GPU память.
3. Произвести вычисление в GPU через функцию ядра.
4. Скопировать вычисленные данные из GPU памяти в ОЗУ.
5. Посмотреть результаты.
6. Высвободить используемые ресурсы.

Функция ядра, которая будет осуществлять сложение векторов:

```
// функция сложения двух векторов
__global__ void addVector(float* left, float* right, float* result)
{
    //Получаем id текущей нити.
    int idx = threadIdx.x;

    //Расчитываем результат.
    result[idx] = left[idx] + right[idx];
}
```

- Распараллеливание будет выполнено автоматически при запуске ядра.
- В этой функции так же используется встроенная переменная **threadIdx** и её поле **x**, которая позволяет задать соответствие между расчетом элемента вектора и нитью в блоке. Делаем расчет каждого элемента вектора в отдельной нити.

```
#define SIZE 512

__host__ int main()
{
    //Выделяем память под вектора
    float* vec1 = new float[SIZE];
    float* vec2 = new float[SIZE];
    float* vec3 = new float[SIZE];

    //Инициализируем значения векторов
    for (int i = 0; i < SIZE; i++)
    {
        vec1[i] = i;
        vec2[i] = i;
    }
}
```

```
//Указатели на память видеокарте  
float* devVec1;  
float* devVec2;  
float* devVec3;  
  
//Выделяем память для векторов на видеокарте  
cudaMalloc((void**)&devVec1, sizeof(float) * SIZE);  
cudaMalloc((void**)&devVec2, sizeof(float) * SIZE);  
cudaMalloc((void**)&devVec3, sizeof(float) * SIZE);  
  
//Копируем данные в память видеокарты  
cudaMemcpy(devVec1, vec1, sizeof(float) * SIZE, cudaMemcpyHostToDevice);  
cudaMemcpy(devVec2, vec2, sizeof(float) * SIZE, cudaMemcpyHostToDevice);  
...  
}
```

Теперь переходим к непосредственному вызову ядра для вычисления на GPU.

```
...  
dim3 gridSize = dim3(1, 1, 1);    //Размер используемого грида  
dim3 blockSize = dim3(SIZE, 1, 1); //Размер используемого блока  
  
//Выполняем вызов функции ядра  
addVector<<<gridSize, blockSize>>>(devVec1, devVec2, devVec3);  
...
```

- Теперь остается скопировать результат расчета из видеопамати в память хоста.
- Но у функций ядра при этом есть особенность – **асинхронное исполнение**, то есть, если после вызова ядра начал работать следующий участок кода, то это ещё не значит, что GPU выполнил расчеты.
- Для завершения работы заданной функции ядра необходимо использовать средства синхронизации, например event.
- Перед копированием результатов на хост выполняем синхронизацию нитей GPU через event.

Код после вызова ядра:

```
//Выполняем вызов функции ядра
addVector<<<blocks, threads>>>(devVec1, devVec2, devVec3);

//Хендл event'a
cudaEvent_t syncEvent;

cudaEventCreate(&syncEvent);      //Создаем event
cudaEventRecord(syncEvent, 0);    //Записываем event
cudaEventSynchronize(syncEvent);  //Синхронизируем event

//Только теперь получаем результат расчета
cudaMemcpy(vec3, devVec3, sizeof(float) * SIZE, cudaMemcpyDeviceToHost);
```

Выводим результат на экран и чистим выделенные ресурсы:

```
//Результаты расчета
for (int i = 0; i < SIZE; i++)
{
    printf("Element #%i: %.1f\n", i , vec3[i]);
}

//
// Высвобождаем ресурсы
//

cudaEventDestroy(syncEvent);

cudaFree(devVec1);
cudaFree(devVec2);
cudaFree(devVec3);

delete[] vec1; vec1 = 0;
delete[] vec2; vec2 = 0;
delete[] vec3; vec3 = 0;
```

Преимущества CUDA

- интерфейс программирования приложений CUDA основан на стандартном языке программирования Си с расширениями, что упрощает процесс изучения и внедрения архитектуры CUDA;
- CUDA обеспечивает доступ к разделяемой между потоками памяти размером в 16 Кб на мультипроцессор, которая может быть использована для организации кэша с широкой полосой пропускания, по сравнению с текстурными выборками;
- более эффективная передача данных между системной и видеопамятью
- отсутствие необходимости в графических API с избыточностью и накладными расходами;
- линейная адресация памяти, возможность записи по произвольным адресам;
- аппаратная поддержка целочисленных и битовых операций.

Основные ограничения CUDA:

- отсутствие поддержки рекурсии для выполняемых функций;
- минимальная ширина блока в 32 потока;
- закрытая архитектура CUDA, принадлежащая Nvidia.